

UNIVERSIDAD DE MANIZALES

Tecnología en Desarrollo de Software para Negocios Digitales

Seguridad de Aplicaciones

Universidad de Manizales

Sergio Pinilla Valencia

Pablo Nieto Pérez

NegocioSeguro (Compra Segura)

Producto 2. Prevención contra la Inyección sql

Informe de Hardening

29 de Octubre de 2026

Tabla de contenido

1. Introducción **4**
2. Objetivos **4**
2.1 Objetivo general 4
2.2 Objetivos específicos 4
3. Justificación **4**
4. Desarrollo **5**
4.1 Alcance y metodología 5
4.2 Estado inicial y preparación del entorno 5
4.3 Implementación de las capas de defensa 6
4.3.1 Mínimo privilegio y configuración de conexion.php 6
4.3.2 Validación por lista blanca y consultas preparadas en registro.php 6
4.3.3 Consultas preparadas y verificación de contraseñas en login.php 7
4.3.4 PDO, parámetros nombrados y manejo de excepciones en buscar_producto.php 7
4.3.5 Validación de tipos, consulta preparada y verificación del precio en pedido.php 8
Evidencias de las pruebas (antes y después) 8
4.4 Matriz de Verificación de Hardening 9
4.5 Análisis de defensa en profundidad 10
4.6 Limitaciones y riesgos residuales 11
4.7 Respuestas a las tres preguntas de auditoría interna 11
Pregunta 1. Análisis de caso en pedido.php 11
Pregunta 2. Evaluación de las capas de defensa 11
Pregunta 3. Desafío de auditoría (límites de la validación) 12
5. Discusión general **12**
6. Conclusiones **12**
7. Bibliografía **13**
8. Anexos **13**

Tabla de figuras

Figura 1. buscar_producto.php con UNION antes, en vez de productos salen los usuarios con las claves en texto plano

Figura 2. buscar_producto.php con UNION después, el mismo payload no trae nada porque se buscó como texto

Figura 3. login.php con la tautología ' OR '1'='1 antes: «Bienvenido, admin. Acceso concedido.»

Figura 4. login.php con la tautología después: «Usuario o clave incorrectos.»

Figura 5. buscar_producto.php con extractvalue después, no sale el error ni la versión del motor

Figura 6. login.php con la ciega booleana después: «Usuario o clave incorrectos.»

Figura 7. registro.php con admin'-- después, lo frena la lista blanca antes de llegar a la base

Figura 8. pedido.php con el precio cambiado a 1 antes, cobra $2

Figura 9. pedido.php con el precio cambiado a 1 después, cobra el real que es $6.000

1. Introducción

Este trabajo es la continuación de la Tarea 4, ahí había auditado la aplicación NegocioSeguro que es la misma Compra Segura que venimos usando en el curso y había reproducido en mi xampp local varios ataques de inyección sql siguiendo el owasp Top 10. En esa tarea el sistema quedó diagnosticado,entonces lo que toca ahora es la otra mitad que es taparle esos huecos y sobre todo comprobar que de verdad quedaron tapados

La idea que seguí es la de defensa en profundidad, que básicamente es no dejarle todo a una sola barrera sino poner varias capas que se respalden entre ellas. Trabajé tres, la primera es darle a la aplicación un usuario de base de datos con los permisos justos (mínimo privilegio), la segunda es validar lo que entra por los formularios con una lista blanca y la tercera es cambiar todas las consultas para que usen sentencias preparadas con parámetros en vez de pegar lo que escribe el usuario dentro del sql. El marco sigue siendo el owasp Top 10:2021, sobre todo la A03 (Inyección), y para la parte de buenas prácticas me apoyé en la ISO/IEC 27002

El informe sigue la plantilla de la materia: primero los objetivosy la justificación, luego el desarrollo que es la parte más larga (el estado inicial, las correcciones archivo por archivo, la matriz de verificación y el análisis), después la discusión y las conclusiones y al final la bibliografía y los anexos con el código. Todo lo hice en local sobre una aplicación que es insegura a propósito, entonces no hay ningún dato real de por medio

2. Objetivos

2.1 Objetivo general

Implementar y verificar en mi entorno local una defensa en profundidad contra la inyección sql en NegocioSeguro, para que los vectores que encontramos en la Clase 6 queden cerrados y comprobados con pruebas que se puedan repetir

2.2 Objetivos específicos

1. Crear un usuario de base de datos con privilegios mínimos (solo SELECT, INSERT y UPDATE sobre la base del proyecto) y que la aplicación se conecte con ese usuario y no con root
2. Refactorizar los cinco archivos (conexion.php, registro.php, login.php, buscar_producto.php y pedido.php) con validación por lista blanca y consultas preparadas con MySQLi y PDO
3. Repetir sobre el sistema ya corregido los mismos payloads de la Clase 6 y anotar el resultado real de cada uno en la Matriz de Verificación de Hardening
4. Explicar por qué las tres capas se complementan y responder las tres preguntas de auditoría interna de la guía, incluyendo hasta dónde llega la validación que implementé

3. Justificación

NegocioSeguro maneja justo lo que un atacante quisiera: los usuarios con sus contraseñas y los pedidos. En la Tarea 4 mostré que con una sola línea en el buscador se podían sacar

todas las claves y que el precio de un pedido se podía cambiar desde el navegador, eso ya quedó diagnosticado, entonces lo que vale ahora es cerrarlo y no quedarme en el «ya lo corregí» sino probar que el ataque dejó de funcionar

Seguí con el owasp porque la A03 es exactamente el fallo que tiene la aplicación y lo complementé con el mínimo privilegio de la ISO/IEC 27002, que es la idea de no darle a un componente más permisos de los que necesita. Hacerlo en local sobre una app vulnerable a propósito me deja ver el daño real y comprobar la corrección sin poner en riesgo a nadie

4. Desarrollo

4.1 Alcance y metodología

El alcance son los cinco archivos de NegocioSeguro y los seis payloads de la matriz de la guía, lo que queda por fuera (otras categorías del owasp, el cifrado de las contraseñas, etc) lo dejo anotado en la parte de limitaciones

Para la metodología tuve en cuenta algo que la guía repite mucho y es que implementar no es lo mismo que verificar. No sirve decir «cambié login.php a consulta preparada entonces ya no es vulnerable», lo que hice en cada punto fue primero confirmar que el ataque funcionaba en el estado original, después aplicar la corrección y por último repetir el mismo payload sobre el código corregido y anotar la respuesta exacta que dio el sistema. Solo cuando esa prueba sale como se espera marco la fila como verificada

Las pruebas las hice contra dos copias, la original (la carpeta compraseguras_backup) para sacar el «antes» y la ya corregida para el «después», todas las figuras salen de ahí

4.2 Estado inicial y preparación del entorno

Fecha y hora de la verificación	20 de octubre de 2026, en mi computador con el entorno local
Estado de los servicios XAMPP	Apache (puerto 8080) y MySQL/MariaDB 10.4.32 prendidos y funcionando
Método de respaldo	Hice dos respaldos: una copia completa de la carpeta del proyecto con el nombre compraseguras_backup y una etiqueta de Git llamada antes-producto2 sobre el commit de antes de los cambios
Ubicación del respaldo	C:\xampp\htdocs\compraseguras_backup y la etiqueta antes-producto2 en el repositorio https://github.com/pabloWIB/CompraSegura
Estado inicial confirmado	Antes de tocar nada repetí los payloads sobre la copia original y todos seguían funcionando: el login se saltaba con la tautología, el buscador soltaba las claves por UNION, el error de extractvalue mostraba la versión del motor (~10.4.32-MariaDB) y el pedido aceptaba el precio que le pusiera el cliente. Esa es la línea base para comparar

La base compraseguras tiene las tres tablas de siempre (usuarios, productos y pedidos) con sus datos de prueba. Un detalle con los nombres: la guía habla de la base «negocioseguro» pero en mi proyecto se llama compraseguras porque es la misma app, entonces todos los

comandos y la conexión usan ese nombre, el usuario restringido si lo dejé como pide la guía, negocioseguro_app

4.3 Implementación de las capas de defensa

Acá va archivo por archivo qué corregí y con qué prueba lo confirmé, las capas no van separadas porque en un mismo archivo se juntan varias

4.3.1 Mínimo privilegio y configuración de conexion.php

Objetivo de seguridad	Que aunque una inyección llegara a ejecutarse la cuenta de base de datos de la app no tenga permiso para hacer un daño mayor como borrar o cambiar tablas
Problema o riesgo mitigado	La app se conectaba como root, que puede hacer de todo en MySQL (A05, configuración incorrecta), entonces cualquier inyección que funcionara heredaba esos permisos
Cambio implementado	Creé el usuario negocioseguro_app con GRANT SELECT, INSERT, UPDATE ON compraseguras.* (sin DROP, ALTER ni DELETE) y cambié conexion.php para que se conecte con él. También el mensaje de error de conexión ya no muestra el detalle técnico
Evidencia técnica	Código completo en el Anexo C
Prueba realizada	Con el usuario nuevo intenté DROP TABLE compraseguras.productos; directo en el motor
Resultado observado	ERROR 1142 (42000): DROP command denied to user 'negocioseguro_app'@'localhost'. Un SELECT normal si responde, o sea la app sigue funcionando
Análisis de eficacia y limitaciones	Esta capa no evita la inyección, lo que hace es contenerla y bajar el daño que puede hacer. Es la última red, no la primera

4.3.2 Validación por lista blanca y consultas preparadas en registro.php

Objetivo de seguridad	Rechazar lo que traiga caracteres peligrosos antes de que llegue a la base de datos y de todas formas insertar con parámetros
Problema o riesgo mitigado	El INSERT pegaba $_POST['usuario'] y $_POST['clave'] directo en la consulta, con una comilla ya se rompía (A03)
Cambio implementado	Validación por lista blanca con preg_match('/^[a-zA-Z0-9_]{3,20}$/', $usuario), una longitud mínima para la clave y el INSERT con prepare() y bind_param("ss", ...)
Evidencia técnica	Figura 7 y Anexo C
Prueba realizada	Registrar el usuario admin'--

Resultado observado	«Nombre de usuario invalido. Use solo letras, numeros y guion bajo (3 a 20 caracteres).» y la solicitud se corta antes de llegar a la base de datos
Análisis de eficacia y limitaciones	Acá trabajan dos capas, la lista blanca frena el valor y si algo pasara la consulta preparada lo tomaría como dato. No cubre que la clave siga en texto plano, eso es de otra entrega

4.3.3 Consultas preparadas y verificación de contraseñas en login.php

Objetivo de seguridad	Que el login no se pueda saltar con una inyección y que la clave no se compare dentro del sql
Problema o riesgo mitigado	La consulta WHERE usuario = '$usuario' AND clave = '$clave' dejaba hacer el bypass con la tautología y la extracción ciega booleana (A03 y A07)
Cambio implementado	Consulta preparada que busca solo por el usuario (WHERE usuario = ?) y la clave se compara después en php. El nombre se imprime con htmlspecialchars()
Evidencia técnica	Figuras 3, 4 y 6 y Anexo C
Prueba realizada	El bypass de siempre ' OR '1'='1 y la extracción ciega admin' AND SUBSTRING(clave,1,1)='a' -- -
Resultado observado	En los dos casos sale «Usuario o clave incorrectos.» porque el motor busca un usuario que se llame literalmente así y como no existe no deja entrar
Análisis de eficacia y limitaciones	La consulta preparada corta la inyección desde la raíz y separar la verificación de la clave en php deja listo el terreno para cifrarla en la próxima entrega

4.3.4 PDO, parámetros nombrados y manejo de excepciones en buscar_producto.php

Objetivo de seguridad	Que el buscador no deje leer otras tablas ni muestre información del motor en los errores
Problema o riesgo mitigado	WHERE nombre LIKE '%$busqueda%' permitía la inyección basada en error (extractvalue) y la de UNION (A03)
Cambio implementado	Lo reescribí con PDO y un parámetro con nombre :busqueda (el % va en el valor y no en el sql), con ERRMODE_EXCEPTION y un catch que muestra un mensaje general. La conexión de PDO también usa el usuario restringido
Evidencia técnica	Figuras 1, 2 y 5 y Anexo C

Prueba realizada	' AND extractvalue(1,concat(0x7e,(SELECT version()))) -- - y inexistente' UNION SELECT id, usuario, clave, 'robado' FROM usuarios -- -
Resultado observado	Ninguno trae datos ni errores, la búsqueda sale vacía porque el payload se busca como texto. Ya no aparece «admin - $admin123» ni la versión del motor
Análisis de eficacia y limitaciones	La consulta preparada cierra las dos variantes al tiempo y no mostrar el detalle de la excepción cierra la fuga de información por los errores

4.3.5 Validación de tipos, consulta preparada y verificación del precio en pedido.php

Objetivo de seguridad	Cerrar la inyección del INSERT y el fraude del precio al mismo tiempo y pedir que haya sesión
Problema o riesgo mitigado	El INSERT pegaba los datos (A03), el precio llegaba de un campo oculto del formulario (A04 y A08) y se podían hacer pedidos sin sesión (A01)
Cambio implementado	Pide $_SESSION['usuario_id'], valida con filter_var(..., FILTER_VALIDATE_INT), vuelve a consultar el precio en la base por el id del producto (ya no lo toma del POST) e inserta con consulta preparada bind_param("iiid", ...)
Evidencia técnica	Figuras 8 y 9 y Anexo C
Prueba realizada	Pedido del producto 1 (precio real $3.000) con el precio cambiado a 1 y cantidad 2, un producto_id que no es entero y un pedido sin iniciar sesión
Resultado observado	Total calculado $6.000,00, o sea el real, ignorando el precio que mandé. El id que no es entero da «Los datos del pedido son invalidos» y sin sesión da «Debe iniciar sesion para realizar un pedido»
Análisis de eficacia y limitaciones	Acá se ve que la consulta preparada sola no alcanza, el fraude del precio es un fallo de diseño y se cierra con la reconsulta en el servidor que es otra medida

Evidencias de las pruebas (antes y después)

Estas son las capturas de las pruebas, primero el ataque funcionando en la copia original y después el mismo payload sobre el código corregido



Figura 1. buscar_producto.php con UNION antes, en vez de productos salen los usuarios con las claves en texto plano



Figura 2. buscar_producto.php con UNION después, el mismo payload no trae nada porque se buscó como texto

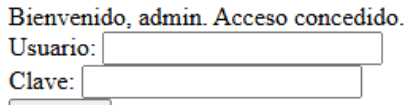


Figura 3. login.php con la tautología ' OR '1'='1 antes: «Bienvenido, admin. Acceso concedido.»

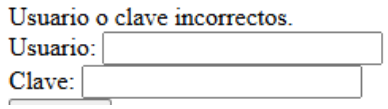


Figura 4. login.php con la tautología después: «Usuario o clave incorrectos.»



Figura 5. buscar_producto.php con extractvalue después, no sale el error ni la versión del motor

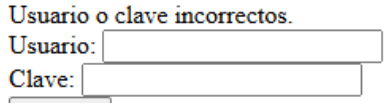


Figura 6. login.php con la ciega booleana después: «Usuario o clave incorrectos.»

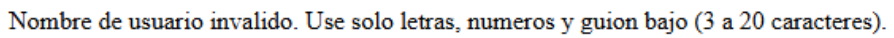


Figura 7. registro.php con admin'-- después, lo frena la lista blanca antes de llegar a la base

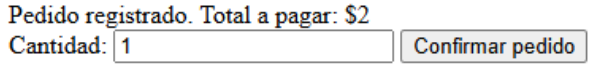


Figura 8. pedido.php con el precio cambiado a 1 antes, cobra $2

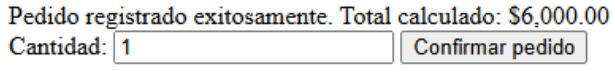


Figura 9. pedido.php con el precio cambiado a 1 después, cobra el real que es $6.000

4.4 Matriz de Verificación de Hardening

En la matriz junté los seis puntos de la guía: el payload de la Clase 6, si funcionaba antes y si funciona ahora, la capa que lo frenó, el resultado sobre el código corregido y la evidencia

#	Archivo	Payload (Clase 6)	Capa que lo neutralizó	Funcionaba antes	Funciona ahora	Resultado observado	Evidencia	Estado
1	login.php	' OR '1'='1	Consulta preparada (MySQLi)	Sí	No	«Usuario o clave incorrectos.»	Fig. 3 y 4	Verificado / Exitoso
2	buscar_producto.php	' AND extractvalue(1,concat(0x7e,(SELECT version()))) -- -	Consulta preparada (PDO) y no mostrar la excepción	Sí	No	No sale error ni la versión, búsqueda vacía	Fig. 5	Verificado / Exitoso
3	buscar_producto.php	inexistente' UNION SELECT id, usuario, clave, 'robado' FROM usuarios -- -	Consulta preparada (PDO)	Sí	No	No salen usuarios ni claves, búsqueda vacía	Fig. 1 y 2	Verificado / Exitoso
4	login.php	admin' AND SUBSTRING(clave,1,1)='a' -- -	Consulta preparada (MySQLi)	Sí	No	«Usuario o clave incorrectos.» y no cambia con la condición	Fig. 6	Verificado / Exitoso
5	registro.php	admin'--	Lista blanca (preg_match) y consulta preparada	Sí	No	«Nombre de usuario invalido» y se corta antes de la base	Fig. 7	Verificado / Exitoso
6	pedido.php	Inyección en el id y precio manipulado	filter_var, reconsulta del precio, consulta preparada y sesión obligatoria	Sí	No	Cobra el precio real ($6.000), rechaza id no entero y pedido sin sesión	Fig. 8 y 9	Verificado / Exitoso

Los seis quedan en «No» en la columna de funciona ahora, cada uno con su prueba repetida y su figura

4.5 Análisis de defensa en profundidad

Las tres capas no son lo mismo repetido tres veces, cada una cubre un momento diferente del ataque. La lista blanca actúa en la entrada, filtra el dato apenas llega y antes de que toque la base. La consulta preparada actúa cuando se consulta la base, aunque pase un valor raro la base lo toma como dato y nunca como parte de la orden sql. Y el mínimo privilegio actúa sobre el daño, si por algún hueco que no vi una inyección llegara a ejecutarse la cuenta de la app no podría borrar ni cambiar tablas

Por eso se complementan y ninguna sobra. Si solo tuviera la lista blanca un caso que no previ se podría colar, si solo tuviera la consulta preparada estaría bien contra la inyección pero la app seguiría corriendo como root y cualquier otro fallo tendría permisos de más, y si solo tuviera el mínimo privilegio la inyección seguiría funcionando solo que haría menos daño. Con las tres juntas un ataque tendría que romperlas todas al tiempo, que es lo que se busca

4.6 Limitaciones y riesgos residuales

Con esto el sistema no queda «seguro del todo» y es mejor decirlo claro:

- La verificación la hice con los payloads conocidos de la Clase 6, puede haber otras combinaciones que no probé y una herramienta automática encontraría casos que esta matriz no tiene
- Quedan categorías del owasp por fuera, la más visible es que las contraseñas siguen en texto plano (A02) y eso se corrige con password_hash y password_verify en otra entrega
- El control de autorización, que un usuario solo vea o cambie lo suyo, es diferente a validar la entrada y la consulta preparada no lo cubre sola
- Las pruebas fueron a mano y en local, no hay registro ni monitoreo de intentos (A09) entonces un ataque lento podría pasar sin que nadie lo note
- Como explico en la pregunta 3, la cantidad se valida por tipo pero no tiene un límite de negocio y un valor gigante pasa

4.7 Respuestas a las tres preguntas de auditoría interna

Pregunta 1. Análisis de caso en pedido.php

En pedido.php había dos fallos diferentes y el cambio los cerró con dos medidas diferentes en el mismo archivo. El primero era la inyección sql, el INSERT pegaba producto_id y cantidad dentro de la consulta y eso se cierra con la consulta preparada (bind_param) que manda esos valores como datos y no como parte del sql. El segundo es de integridad y la app lo venía arrastrando desde la Clase 2, el precio llegaba de un campo oculto del formulario entonces el cliente lo podía poner en 1 y pagar lo que quisiera. Eso no lo arregla ninguna consulta preparada porque el dato llegaba «bien formado», el problema era confiar en él, y se cierra con una medida de diseño que es no leer el precio del POST sino volverlo a consultar en la base por el id del producto, que es la única fuente confiable

La prueba lo confirma, mandando precio 1 y cantidad 2 de un producto de $3.000 el sistema cobró $6.000 que es lo real y no $2. O sea que un mismo cambio tapó una inyección y un fraude de precio pero con herramientas diferentes para cada uno

Pregunta 2. Evaluación de las capas de defensa

Si me tuviera que quedar con una sola me quedo con las consultas preparadas, porque atacan la causa del problema que es mezclar el código con los datos. La consulta preparada separa la orden sql de los valores, entonces frena cualquier variante de inyección (tautología, UNION, basada en error, ciega) sin importar lo que escriba el usuario y lo hace en todos los campos de esa consulta de una vez

La lista blanca sirve mucho pero es campo por campo y depende de que uno haya pensado bien el patrón, si un caso se escapa pasa. Y el mínimo privilegio no evita la inyección, solo baja el daño cuando ya pasó. Para mí las consultas preparadas son la prevención amplia, la lista blanca es una prevención puntual y el mínimo privilegio es contención, entonces si solo puedo tener una barrera me quedo con la que de verdad impide que el ataque funcione

Pregunta 3. Desafío de auditoría (límites de la validación)

Esto lo probé de verdad para no contestar de memoria. Si en cantidad mando 999999999 la consulta preparada no evita el pedido y tiene sentido, porque 999999999 es un entero válido y pasa el filter_var(FILTER_VALIDATE_INT) sin problema, las consultas preparadas protegen contra la inyección pero no contra datos absurdos que vienen bien escritos

El pedido se registró y ahí salió un problema real: php calculó un total de $2.999.999.997.000 pero la columna total es DECIMAL(10,2) que guarda máximo 99.999.999,99, y como el sql_mode de este MariaDB no es estricto la base no dio error sino que cortó el valor sin avisar y guardó 99.999.999,99, que no es lo que se le mostró al usuario. O sea que el daño no es de inyección, es de integridad y de negocio: un pedido sin sentido, un total mal guardado y la posible venta de un stock que no existe

Se arregla con una regla de negocio, validar un rango normal para la cantidad (entre 1 y un máximo o contra el stock del producto) y revisar que el total quepa en la columna, y también conviene activar el modo estricto de la base para que un desborde falle en vez de cortarse callado

5. Discusión general

Viendo todo junto, lo primero es que las cinco inyecciones venían del mismo error que es pegar lo que escribe el usuario dentro del sql, y una sola práctica que son las consultas preparadas las cerró todas. Eso deja clara la prioridad, antes de meterse con controles más finos lo que más sirve es parametrizar todas las consultas porque de una vez tapa los cinco vectores

Lo segundo es que no todo se arregla con consultas preparadas y pedido.php lo deja claro: el fraude del precio y pedir sin sesión son fallos de diseño y la cantidad gigante de la pregunta 3 es un fallo de reglas de negocio, ninguno lo arregla la parametrización y necesitan controles aparte (volver a consultar el precio, pedir sesión, validar rangos). Por eso tiene sentido la defensa en profundidad, cada capa cubre algo que las otras no

Si alguien dijera que con las consultas preparadas ya está todo, la respuesta está en la matriz y en la pregunta 3: la inyección si quedó cerrada y verificada pero el total cortado muestra que quedan problemas que la parametrización no toca, y decir «ya es seguro» sería justo la conclusión exagerada que la guía pide no hacer

6. Conclusiones

- Implementé las tres capas y comprobé con pruebas repetidas que los seis payloads de la Clase 6 dejaron de funcionar sobre el código corregido, los seis quedan en «No» en la matriz de la sección 4.4 (objetivos 1, 2 y 3)

- El mínimo privilegio lo probé aparte, el usuario negocioseguro_app no puede hacer DROP pero la app sigue funcionando con SELECT, INSERT y UPDATE que es lo que necesita
- Las tres capas se complementan: la lista blanca filtra la entrada, la consulta preparada separa el código de los datos y el mínimo privilegio contiene el daño, cada una cubre un momento diferente del ataque (objetivo 4)
- La verificación se limitó a los payloads conocidos y en local, no digo que el sistema esté libre de otras inyecciones que no probé ni que otras categorías del owasp como las contraseñas en texto plano queden resueltas con esta entrega
- La pregunta 3 dejó un hallazgo que confirma lo anterior, la consulta preparada no frena una cantidad de 999.999.999 que pasa la validación de tipo y termina guardando un total cortado, eso necesita una validación de negocio aparte

7. Bibliografía

International Organization for Standardization. (2013). ISO/IEC 27002:2013. Information technology, Security techniques, Code of practice for information security controls. ISO

OWASP Foundation. (s.f.). OWASP Top 10. Recuperado el 20 de octubre de 2026 de https://owasp.org/www-project-top-ten/

OWASP Foundation. (s.f.). SQL Injection Prevention Cheat Sheet. Recuperado el 20 de octubre de 2026 de https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

The PHP Group. (s.f.). Sentencias preparadas con MySQLi. Manual de PHP. Recuperado el 20 de octubre de 2026 de https://www.php.net/manual/es/mysqli.quickstart.prepared-statements.php

The PHP Group. (s.f.). PDO::prepare. Manual de PHP. Recuperado el 20 de octubre de 2026 de https://www.php.net/manual/es/pdo.prepare.php

8. Anexos

Anexo A. Evidencias complementarias: las nueve figuras salen de las pruebas reales del 20/10/2026 y las imágenes en tamaño completo están en la carpeta tareas/producto2-imagenes del repositorio, con el prefijo antes- y despues-

Anexo B. Resumen de cambios por archivo: conexion.php usa el usuario restringido en vez de root, registro.php tiene lista blanca e INSERT preparado, login.php hace el SELECT preparado por usuario y compara la clave en php, buscar_producto.php usa PDO con parámetro con nombre y manejo de excepciones y pedido.php valida tipos, vuelve a consultar el precio y hace el INSERT preparado

Anexo C. Código completo de los cinco archivos, también van en el ZIP de la actividad

conexion.php

```php
<?php
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

$host = "localhost";
$user = "negocioseguro_app";
$pass = "ClaveSegura123!";
$db   = "compraseguras";

$conn = new mysqli($host, $user, $pass, $db);

if ($conn->connect_error) {
    die("Error critico de conexion a la base de datos.");
}
?>
```

registro.php

```php
<?php
include 'conexion.php';
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $usuario = trim($_POST['usuario']);
    $clave   = $_POST['clave'];

    if (!preg_match('/^[a-zA-Z0-9_]{3,20}$/', $usuario)) {
        die("Nombre de usuario invalido. Use solo letras, numeros y guion bajo (3 a 20
caracteres).");
    }

    if (strlen($clave) < 8) {
        die("La clave debe tener al menos 8 caracteres.");
    }

    $stmt = $conn->prepare("INSERT INTO usuarios (usuario, clave) VALUES (?, ?)");
    $stmt->bind_param("ss", $usuario, $clave);

    if ($stmt->execute()) {
        echo "Usuario registrado con exito.";
    } else {
        echo "No se pudo registrar el usuario.";
    }
    $stmt->close();
}
?>
<form method="POST">
  Nuevo usuario: <input type="text" name="usuario"><br>
  Clave: <input type="password" name="clave"><br>
  <button type="submit">Registrarme</button>
</form>
```

login.php

```php
<?php
include 'conexion.php';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $usuario = $_POST['usuario'];
    $clave   = $_POST['clave'];

    $stmt = $conn->prepare("SELECT * FROM usuarios WHERE usuario = ?");
    $stmt->bind_param("s", $usuario);
    $stmt->execute();
    $resultado = $stmt->get_result();

    if ($resultado && $resultado->num_rows > 0) {
        $fila = $resultado->fetch_assoc();
        if ($clave === $fila['clave']) {
            $_SESSION['usuario_id'] = $fila['id'];
            $_SESSION['usuario']    = $fila['usuario'];
            echo "Bienvenido, " . htmlspecialchars($fila['usuario']) . ". Acceso
concedido.";
        } else {
            echo "Usuario o clave incorrectos.";
        }
    } else {
        echo "Usuario o clave incorrectos.";
    }
    $stmt->close();
}
?>
<form method="POST">
  Usuario: <input type="text" name="usuario"><br>
  Clave: <input type="password" name="clave"><br>
  <button type="submit">Ingresar</button>
</form>
```

buscar_producto.php

```php
<?php
session_start();
$busqueda = isset($_GET['q']) ? trim($_GET['q']) : '';

if ($busqueda !== '') {
    try {
        $pdo = new PDO(
            "mysql:host=localhost;dbname=compraseguras;charset=utf8mb4",
            "negocioseguro_app",
            "ClaveSegura123!"
        );
        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

        $stmt = $pdo->prepare("SELECT nombre, precio FROM productos WHERE nombre LIKE
:busqueda");
        $stmt->bindValue(':busqueda', '%' . $busqueda . '%');
        $stmt->execute();

        while ($fila = $stmt->fetch(PDO::FETCH_ASSOC)) {
            echo htmlspecialchars($fila['nombre']) . " - $" .
htmlspecialchars($fila['precio']) . "<br>";
        }
    } catch (PDOException $e) {
        echo "Ocurrio un error al procesar la busqueda. Intente nuevamente.";
    }
}
?>
<form method="GET">
  Buscar producto: <input type="text" name="q">
  <button type="submit">Buscar</button>
</form>
```

pedido.php

```php
<?php
include 'conexion.php';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $producto_id = $_POST['producto_id'] ?? '';
    $cantidad    = $_POST['cantidad'] ?? '';
    $usuario_id  = $_SESSION['usuario_id'] ?? 0;

    if ($usuario_id === 0) {
        die("Error: Debe iniciar sesion para realizar un pedido.");
    }

    if (!filter_var($producto_id, FILTER_VALIDATE_INT) ||
        !filter_var($cantidad, FILTER_VALIDATE_INT) ||
        $cantidad <= 0) {
        die("Error: Los datos del pedido son invalidos.");
    }

    $stmt_precio = $conn->prepare("SELECT precio FROM productos WHERE id = ?");
    $stmt_precio->bind_param("i", $producto_id);
    $stmt_precio->execute();
    $res_precio = $stmt_precio->get_result();

    if ($res_precio->num_rows === 0) {
        die("Error: El producto seleccionado no existe.");
    }

    $producto    = $res_precio->fetch_assoc();
    $precio_real = $producto['precio'];
    $total       = $precio_real * $cantidad;
    $stmt_precio->close();

    $stmt_pedido = $conn->prepare("INSERT INTO pedidos (usuario_id, producto_id,
cantidad, total) VALUES (?, ?, ?, ?)");
    $stmt_pedido->bind_param("iiid", $usuario_id, $producto_id, $cantidad, $total);

    if ($stmt_pedido->execute()) {
        echo "Pedido registrado exitosamente. Total calculado: $" . number_format($total,
2);
    } else {
        echo "Error al registrar el pedido.";
    }
    $stmt_pedido->close();
}
?>
<form method="POST">
  <input type="hidden" name="producto_id" value="1">
  <input type="hidden" name="precio" value="3000">
  Cantidad: <input type="number" name="cantidad" value="1">
  <button type="submit">Confirmar pedido</button>
</form>
```